

Лабораторна робота 1: Дослідження архітектури розподіленої системи

1. Запуск системи

Стан на цій машині під час перевірки: - PostgreSQL активний.
- Backend запускається на 0.0.0.0:8080. - API працює коректно (health, list, create, patch, delete).

Перевірка підключення до БД (реальний результат):

current_user = todouser current_database = tododb

Скріншот: локально працююча система

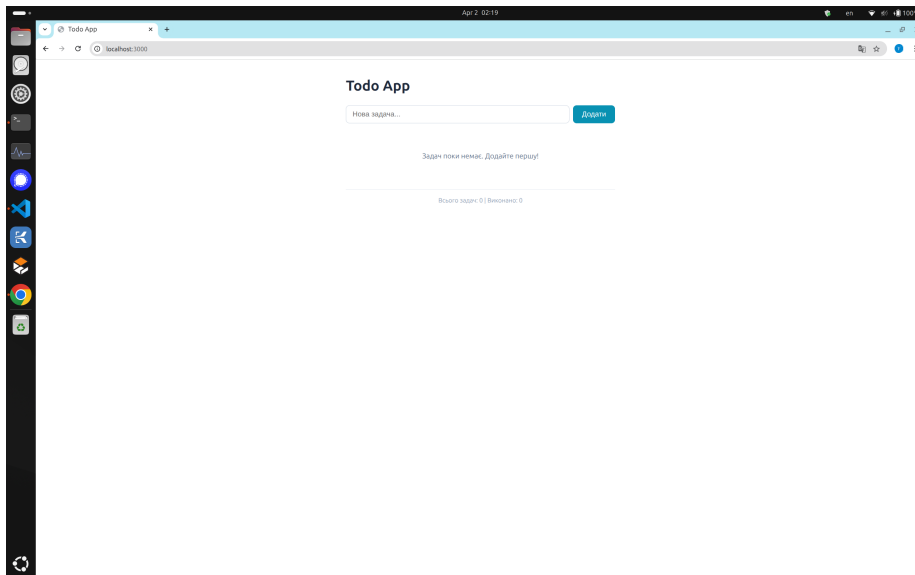


Figure 1: Локально працююча система

2. Дослідження архітектури

2.1. Яку адресу і порт слухає кожен компонент

- Frontend (Vite dev server): порт 3000 (через VITE_PORT, за замовчуванням 3000).
- Backend (FastAPI/Uvicorn): host 0.0.0.0, порт 8080 (через APP_HOST/APP_PORT).
- PostgreSQL: localhost:5432 (через DATABASE_URL).

2.2. Як frontend знаходить backend

Frontend використовує Vite проху у конфігурації: - всі запити на /api проксуються на VITE_BACKEND_URL - за замовчуванням VITE_BACKEND_URL = http://localhost:8080

У коді React використовується відносний шлях /api/todos, а реальна адреса backend підставляється проксі-рівнем Vite.

2.3. Як backend підключається до БД

Параметри підключення беруться з DATABASE_URL у backend/.env (або значення за замовчуванням у config.py):

postgresql://todouser:todopass@localhost:5432/tododb

Backend створює SQLAlchemy engine через create_engine(settings.database_url).

2.4. Протокол між frontend і backend

- HTTP/1.1
- Методи: GET, POST, PATCH, DELETE

2.5. Формат даних

- JSON (Content-Type: application/json)

2.6. Ендпоінти backend

- GET /api/health: перевірка доступності API.
- GET /api/todos: отримати список задач (спадання за created_at).
- GET /api/todos/{todo_id}: отримати одну задачу за id.
- POST /api/todos: створити нову задачу.
- PATCH /api/todos/{todo_id}: оновити title і/або completed.
- DELETE /api/todos/{todo_id}: видалити задачу.

Діаграма архітектури

flowchart LR

U[Browser]

F[Frontend: Vite React\n127.0.0.1:3000]

B[Backend: FastAPI Uvicorn\n0.0.0.0:8080]

D[(PostgreSQL\nlocalhost:5432)]

U -->|HTTP, HTML/CSS/JS| F

```
F -->|HTTP, JSON /api/*| B
B -->|SQL over TCP| D
```

3. Мережева взаємодія

Команди для прямої перевірки через curl:

```
curl -i http://localhost:8080/api/health
```

```
curl -i http://localhost:8080/api/todos
```

```
curl -i -X POST http://localhost:8080/api/todos \
  -H "Content-Type: application/json" \
  -d '{"title":"Lab1 todo"}'
```

```
curl -i -X PATCH http://localhost:8080/api/todos/1 \
  -H "Content-Type: application/json" \
  -d '{"completed":true}'
```

```
curl -i -X DELETE http://localhost:8080/api/todos/1
```

Фактичні відповіді сервера (заголовки + тіло):

```
=== GET /api/health ===
```

```
HTTP/1.1 200 OK
```

```
content-type: application/json
```

```
{"status":"ok"}
```

```
=== GET /api/todos (before) ===
```

```
HTTP/1.1 200 OK
```

```
content-type: application/json
```

```
[]
```

```
=== POST /api/todos ===
```

```
HTTP/1.1 201 Created
```

```
content-type: application/json
```

```
{"id":1,"title":"Lab1 verification task","completed":false,"created_at":"2026-04-02T00:00:00Z"}
```

```
=== PATCH /api/todos/{id} ===
```

```
HTTP/1.1 200 OK
```

```
content-type: application/json
```

```
{"id":1,"title":"Lab1 verification task","completed":true,"created_at":"2026-04-02T00:00:00Z"}
```

```
=== DELETE /api/todos/{id} ===  
HTTP/1.1 204 No Content
```

```
=== GET /api/todos (after) ===  
HTTP/1.1 200 OK  
content-type: application/json
```

```
[]
```

Скріншот: Network з деталями запиту

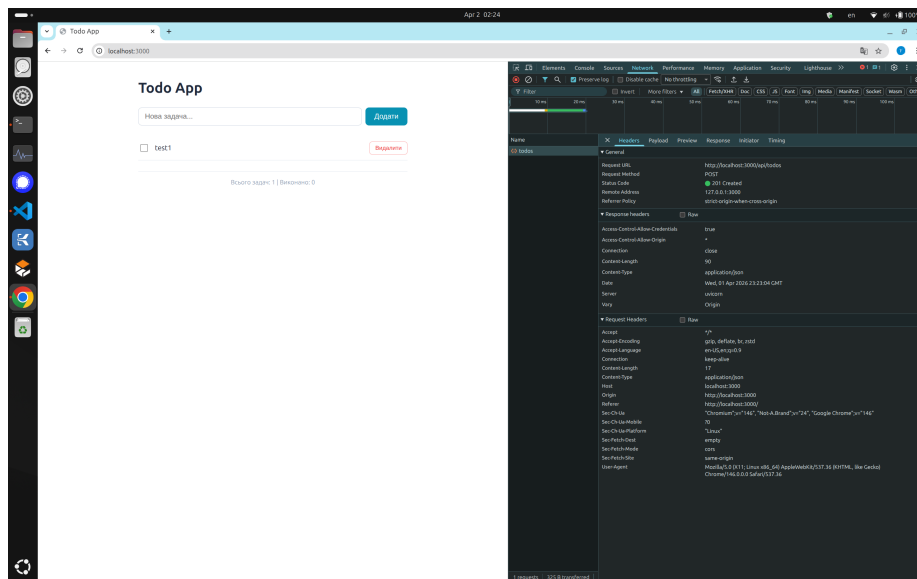


Figure 2: Network з деталями запиту

3.1.

- 0.0.0.0: слухає на всіх мережевих інтерфейсах хоста.
- 127.0.0.1: loopback, тільки локальні підключення з тієї ж машини.
- localhost: DNS-ім'я, зазвичай резолвиться в 127.0.0.1 (інколи також ::1).

3.2.

Локально система працюватиме і при 127.0.0.1, якщо frontend звертається до localhost/127.0.0.1.

3.3.

Інший комп'ютер мережі не зможе підключитися до backend, якщо backend слухає 127.0.0.1.

4. Відмови (аналіз)

4.1. Зупинений backend

- На frontend: помилки HTTP запитів (типово network error/failed to fetch).
- Користувач не може отримати/змінити задачі.

Фактична перевірка:

```
curl -i http://localhost:8080/api/health
curl: (7) Failed to connect to localhost port 8080: Couldn't connect to server
```

4.2. Backend працює, але БД зупинена

- У цьому проєкті backend взагалі не стартує, бо при старті викликається `Base.metadata.create_all(bind=engine)`, що потребує доступної БД.
- Фактичний результат: `OperationalError, connection refused` до 127.0.0.1:5432.

4.3. Backend на іншому порті

- Frontend перестане працювати з API, якщо не оновити `VITE_BACKEND_URL`.
- Потрібно змінити `frontend/.env`: `VITE_BACKEND_URL=http://localhost:` і перезапустити frontend.

Фактична перевірка (backend запущено на 8090):

```
request to old port 8080 -> curl: (7) Failed to connect
request to new port 8090 -> HTTP/1.1 200 OK
```

4.4. Два backend-процеси на різних портах

- Обидва можуть підключатися до однієї PostgreSQL БД за однакового `DATABASE_URL`.
- Це корисно для масштабування (кілька інстансів API за балансувальником) та підвищення доступності.